DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 2 – Controllers, validação, filtros e interfaces Razor reutilizáveis

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** Controllers, validação, filtros e interfaces Razor reutilizáveis	**Relatório 2**

Unidade 2 – MVC e Razor

Objetivo: Construir uma interface MVC para cadastrar e consultar tarefas, usando validação, filtros, layouts, partials e Tag Helpers.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); código desta unidade no commit `e6a3427`. Mesmo ambiente do Relatório 1 (SDK .NET 10.0.114, runtime 10.0.14, xUnit 2.9.3). Execução: `dotnet run --project src/PortalTarefas.Web` e acesso a `https://localhost:7080/`. Capturas geradas com Playwright (`scripts/capturas/atividade2.js`).

2. Resumo teórico aplicado ao projeto

Model, View e Controller. O *controller* recebe a requisição, chama o serviço e escolhe a resposta; o *model* carrega os dados (aqui separados em modelo de domínio `Tarefa` e *ViewModels*); a *view* apenas apresenta. No `TarefasController` cada ação tem poucas linhas: valida, chama `ITarefaServico` e devolve `View(...)` ou `RedirectToAction(...)` (padrão *Post/Redirect/Get*, que evita reenvio do formulário ao atualizar a página).

Binding e validação. O *model binding* preenche `TarefaInputModel` a partir dos campos do formulário. As Data Annotations (`[Required]`, `[StringLength]`, mensagens em português) e a regra customizada em `IValidatableObject` alimentam o `ModelState`; o controller só grava quando `ModelState.IsValid`. Detalhe observado na prática: o MVC só executa a validação de nível de classe (`Validate`) quando as propriedades já são válidas — por isso o erro de prazo aparece depois que o título é corrigido (Figuras B2 e B3). Uma validação **assíncrona dependente de dados** (título duplicado) é feita no controller, pois depende do serviço.

Filtros. Executam em pontos fixos do pipeline MVC: autorização (antes de tudo, decide acesso), recurso, **ação** (antes/depois da ação, com acesso aos argumentos e ao resultado), exceção e resultado. O risco de uso indevido é esconder regra de negócio em filtros (difícil de testar e de enxergar) ou duplicar validação. O `RegistroAcaoFilter` apenas mede e registra ação, duração, validade do `ModelState` e resultado.

Razor e views fortemente tipadas. Razor mistura HTML e C# (`@model`, `@if`, `@foreach`) e é compilado; com `@model TarefaListaViewModel` o compilador verifica nomes de propriedades e o IntelliSense funciona.

Reutilização. `_Layout.cshtml` define cabeçalho, navegação, área de mensagens e rodapé de todas as páginas; a *partial* `_TarefaResumo` é usada na lista e nos detalhes; a partial `_TarefaCampos` é compartilhada por cadastro e edição; o *View Component* `TarefasVencidas` tem lógica própria (consulta o serviço) e aparece no layout; os *Tag Helpers* (`asp-for`, `asp-validation-for`, `asp-action`, `asp-route-id`) geram `name`, `id`, valores, URLs e o token antifalsificação.

Domínio × DTO × ViewModel. A entidade `Tarefa` não chega à view: `TarefaInputModel` expõe só o que o usuário pode digitar (impede *overposting* de `Id`/`CriadaEm`), e `TarefaResumoViewModel` traz campos calculados (`Vencida`).

3. Código-fonte e evidências

Tabela 1 – Arquivos produzidos

Artefato	Responsabilidade
`Controllers/TarefasController.cs`	Ações `Index`, `Details`, `Create` (GET/POST), `Edit` (GET/POST); `[ServiceFilter<RegistroAcaoFilter>]`
`ViewModels/TarefaInputModel.cs`	Data Annotations + `IValidatableObject` (Alta exige prazo; prazo passado só se concluída)
`ViewModels/TarefaViewModels.cs`	`TarefaResumoViewModel`, `TarefaListaViewModel`, `TarefaDetalhesViewModel`, `TarefaEdicaoViewModel`
`Filtros/RegistroAcaoFilter.cs`	Filtro de ação assíncrono (log de ação, duração, ModelState e resultado)
`Servicos/ITarefaServico.cs` + `TarefaServicoMemoria.cs`	Contrato assíncrono e implementação em memória (isola a persistência futura)
`Views/Shared/_Layout`, `_TarefaResumo`, `_TarefaCampos`	Layout comum e partials reutilizadas
`ViewComponents/TarefasVencidasViewComponent.cs`	Componente do layout com tarefas vencidas

Escopo do filtro: registrado por atributo `[ServiceFilter<RegistroAcaoFilter>]` no `TarefasController`, e não globalmente, porque o objetivo é auditar a interface MVC; o controller de diagnóstico da unidade 1 não precisa desse log. Por ser `ServiceFilter`, o filtro recebe `ILogger` por injeção. O trecho abaixo mostra o filtro antes e depois de cada ação; note que, no POST inválido, ele apenas registra `ModelState válido: False` — quem decide é o controller:

```
01:15:26 info RegistroAcaoFilter          corr=e66fd24673d9 [Filtro] Concluído Tarefas.Details [GET] ->
View(NaoEncontrada) status 404 em 0.7 ms
01:15:26 info RequestLoggingMiddleware    corr=e66fd24673d9 [2-Log] saída GET /Tarefas/Details/999 ->
404 em 9.9 ms
01:15:27 info RegistroAcaoFilter          corr=0f14d952edaa [Filtro] Executando Tarefas.Create [POST]
(ModelState válido: False)
01:15:27 info RegistroAcaoFilter          corr=0f14d952edaa [Filtro] Concluído Tarefas.Create [POST] ->
View(padrão) status 200 em 1.2 ms
01:15:27 info RegistroAcaoFilter          corr=21f1ff70fd3d [Filtro] Executando Tarefas.Create [POST]
(ModelState válido: False)
01:15:27 info RegistroAcaoFilter          corr=21f1ff70fd3d [Filtro] Concluído Tarefas.Create [POST] ->
View(padrão) status 200 em 0.7 ms
01:15:27 info RegistroAcaoFilter          corr=dcb00e2e7b9d [Filtro] Executando Tarefas.Create [POST]
(ModelState válido: False)
01:15:27 info RegistroAcaoFilter          corr=dcb00e2e7b9d [Filtro] Concluído Tarefas.Create [POST] ->
View(padrão) status 200 em 0.2 ms
01:15:27 info RegistroAcaoFilter          corr=2f21a3873f32 [Filtro] Executando Tarefas.Create [POST]
(ModelState válido: True)
01:15:27 info RegistroAcaoFilter          corr=2f21a3873f32 [Filtro] Concluído Tarefas.Create [POST] ->
View(padrão) status 200 em 0.2 ms
01:15:27 info RegistroAcaoFilter          corr=c47b9f3d7160 [Filtro] Executando Tarefas.Create [POST]
(ModelState válido: True)
01:15:27 info RegistroAcaoFilter          corr=c47b9f3d7160 [Filtro] Concluído Tarefas.Create [POST] ->
Redirect -> Details em 1.4 ms
```

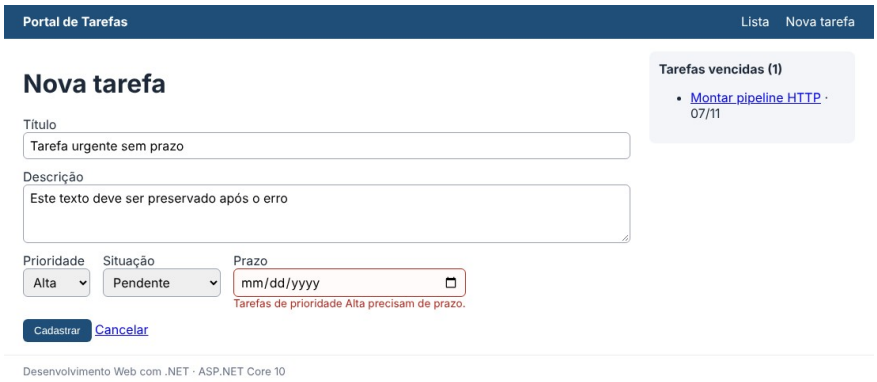


Figura 1 – Validação customizada envolvendo prioridade e prazo; os dados digitados foram preservados

4. Testes e validações obrigatórias

Tabela 2 – Testes obrigatórios (14 testes: 4 de validação, 4 do controller, 6 de páginas)

Caso exigido	Resultado obtido	Evidência
Cadastro válido e com título vazio	Válido: 302 → Detalhes com mensagem. Vazio: a view volta com "Informe o título da tarefa." e a descrição preservada	Fig. B2, B6; `Post_com_titulo_vazio_*`, `Create_valido_*`
Validação customizada de prazo	Alta sem prazo e prazo no passado rejeitados; prazo passado aceito se `Concluida`	Fig. 1, B3; 4 testes unitários
Identificador existente e inexistente	`/Tarefas/Details/2` → 200; `/Tarefas/Details/999` → view `NaoEncontrada` com status 404	Fig. B1; log do filtro; `Details_inexistente_*`
Layout consistente em ≥ 3 páginas	Lista, detalhes, cadastro e edição contêm o mesmo cabeçalho e o View Component	`Paginas_compartilham_layout` (4 páginas)
Filtro sem duplicar regra de negócio	Filtro só registra; validação permanece no ModelState/controller	Log do filtro (seção 3)

5. Aprimoramentos

- **View Component `TarefasVencidas`** no layout, alimentado por `ITarefaServico.ListarVencidasAsync`.
- **Validação assíncrona no servidor dependente de dados:** título duplicado (comparação sem diferenciar maiúsculas) é rejeitado no cadastro e na edição (Fig. B4).
- **Testes unitários** do controller (instância direta com serviço em memória e relógio fixo) e da validação customizada (`Validator.TryValidateObject` com data de referência injetável), além de testes de integração das páginas.
- **Proteção CSRF** global (`AutoValidateAntiforgeryTokenAttribute`): o teste `Post_sem_token_antiforgery_e_rejeitado` confirma 400 sem o token gerado pelo Tag Helper `<form>`.

Referências

MICROSOFT. **Overview of ASP.NET Core MVC**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/overview. Acesso em: 9 nov. 2026.

MICROSOFT. **Model validation in ASP.NET Core MVC and Razor Pages**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/models/validation. Acesso em: 9 nov. 2026.

MICROSOFT. **Filters in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/controllers/filters. Acesso em: 9 nov. 2026.

MICROSOFT. **Layout in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/views/layout. Acesso em: 9 nov. 2026.

MICROSOFT. **Partial views in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/views/partial. Acesso em: 9 nov. 2026.

MICROSOFT. **View components in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/views/view-components. Acesso em: 9 nov. 2026.

MICROSOFT. **Tag Helpers in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Web/Controllers/TarefasController.cs` (commit `e6a3427`)

```
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Filtros;
using PortalTarefas.Web.Modelos;
using PortalTarefas.Web.Servicos;
using PortalTarefas.Web.ViewModels;

namespace PortalTarefas.Web.Controllers;

/// <summary>Interface MVC de tarefas. O controller apenas orquestra: valida, chama o serviço e escolhe a
view.</summary>
[ServiceFilter<RegistroAcaoFilter>]
public sealed class TarefasController(ITarefaServico servico, TimeProvider relogio) : Controller
{
    private DateOnly Hoje => DateOnly.FromDateTime(relogio.GetLocalNow().Date);

    [HttpGet]
    public async Task<IActionResult> Index(SituacaoTarefa? situacao, string? busca, CancellationToken ct)
    {
        var tarefas = await servico.ListarAsync(situacao, busca, ct);
        return View(new TarefaListaViewModel
        {
            Situacao = situacao,
            Busca = busca,
            Itens = tarefas.Select(t => TarefaResumoViewModel.De(t, Hoje)).ToList()
        });
    }

    [HttpGet]
    public async Task<IActionResult> Details(int id, CancellationToken ct)
    {
        var tarefa = await servico.ObterAsync(id, ct);
        if (tarefa is null) return NaoEncontrada(id);
        return View(new TarefaDetalhesViewModel
        {
            Resumo = TarefaResumoViewModel.De(tarefa, Hoje),
            Descricao = tarefa.Descricao,
            CriadaEm = tarefa.CriadaEm
        });
    }

    [HttpGet]
    public IActionResult Create() => View(new TarefaInputModel());

    [HttpPost]
    public async Task<IActionResult> Create(TarefaInputModel entrada, CancellationToken ct)
    {
        await ValidarTituloUnicoAsync(entrada.Titulo, null, ct);
        if (!ModelState.IsValid) return View(entrada);   // preserva o que foi digitado

        var tarefa = new Tarefa();
        entrada.AplicarEm(tarefa);
        var criada = await servico.CriarAsync(tarefa, ct);
        TempData["Mensagem"] = $"Tarefa \"{criada.Titulo}\" cadastrada.";
        return RedirectToAction(nameof(Details), new { id = criada.Id });   // padrão PRG
    }

    [HttpGet]
    public async Task<IActionResult> Edit(int id, CancellationToken ct)
    {
        var tarefa = await servico.ObterAsync(id, ct);
        if (tarefa is null) return NaoEncontrada(id);
        return View(new TarefaEdicaoViewModel { Id = id, Entrada = TarefaInputModel.De(tarefa) });
    }

    [HttpPost]
    public async Task<IActionResult> Edit(int id, [Bind(Prefix = "Entrada")] TarefaInputModel entrada,
CancellationToken ct)
    {
        var tarefa = await servico.ObterAsync(id, ct);
        if (tarefa is null) return NaoEncontrada(id);

        await ValidarTituloUnicoAsync(entrada.Titulo, id, ct);
        if (!ModelState.IsValid) return View(new TarefaEdicaoViewModel { Id = id, Entrada = entrada });

        entrada.AplicarEm(tarefa);
        await servico.AtualizarAsync(tarefa, ct);
        TempData["Mensagem"] = "Alterações salvas.";
        return RedirectToAction(nameof(Details), new { id });
    }
```

```
    /// <summary>Validação assíncrona dependente de dados: o título não pode repetir outro
existente.</summary>
    private async Task ValidarTituloUnicoAsync(string? titulo, int? idAtual, CancellationToken ct)
    {
        if (string.IsNullOrWhiteSpace(titulo)) return;
        if (await servico.ExisteTituloAsync(titulo, idAtual, ct))
        {
            var chave = idAtual is null ? nameof(TarefaInputModel.Titulo) : $"Entrada.
{nameof(TarefaInputModel.Titulo)}";
            ModelState.AddModelError(chave, "Já existe uma tarefa com este título.");
        }
    }

    private ViewResult NaoEncontrada(int id)
    {
        var view = View("NaoEncontrada", id);
        view.StatusCode = StatusCodes.Status404NotFound;
        return view;
    }
}
```

Arquivo: `src/PortalTarefas.Web/ViewModels/TarefaInputModel.cs` (commit `e6a3427`)

```
using System.ComponentModel.DataAnnotations;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.ViewModels;

/// <summary>
/// Dados digitados no formulário de cadastro/edição. Separa a entrada do usuário do
/// modelo de domínio, impedindo overposting de campos como Id e CriadaEm.
/// </summary>
public sealed class TarefaInputModel : IValidatableObject
{
    [Required(ErrorMessage = "Informe o título da tarefa.")]
    [StringLength(120, MinimumLength = 3, ErrorMessage = "O título deve ter entre {2} e {1}
caracteres.")]
    [Display(Name = "Título")]
    public string Titulo { get; set; } = "";

    [StringLength(1000, ErrorMessage = "A descrição deve ter no máximo {1} caracteres.")]
    [Display(Name = "Descrição")]
    [DataType(DataType.MultilineText)]
    public string? Descricao { get; set; }

    [Required(ErrorMessage = "Selecione a prioridade.")]
    [Display(Name = "Prioridade")]
    public PrioridadeTarefa? Prioridade { get; set; } = PrioridadeTarefa.Media;

    [Display(Name = "Situação")]
    public SituacaoTarefa Situacao { get; set; } = SituacaoTarefa.Pendente;

    [DataType(DataType.Date)]
    [Display(Name = "Prazo")]
    public DateOnly? Prazo { get; set; }

    /// <summary>Data de referência para a validação (injetável nos testes).</summary>
    internal DateOnly Hoje { get; set; } = DateOnly.FromDateTime(DateTime.Today);

    /// <summary>
    /// Regra que envolve mais de um campo: tarefas de prioridade Alta exigem prazo,
    /// e o prazo de uma tarefa ainda não concluída não pode estar no passado.
    /// </summary>
    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (Prioridade == PrioridadeTarefa.Alta && Prazo is null)
            yield return new ValidationResult("Tarefas de prioridade Alta precisam de prazo.",
[nameof(Prazo)]);

        if (Prazo is { } prazo && prazo < Hoje && Situacao != SituacaoTarefa.Concluida)
            yield return new ValidationResult("O prazo não pode estar no passado para tarefas não
concluídas.", [nameof(Prazo)]);
    }

    public static TarefaInputModel De(Tarefa t) => new()
    {
        Titulo = t.Titulo, Descricao = t.Descricao, Prioridade = t.Prioridade, Situacao = t.Situacao,
Prazo = t.Prazo
    };

    public void AplicarEm(Tarefa t)
    {
        t.Titulo = Titulo.Trim();
```

```
        t.Descricao = string.IsNullOrWhiteSpace(Descricao) ? null : Descricao.Trim();
        t.Prioridade = Prioridade ?? PrioridadeTarefa.Media;
        t.Situacao = Situacao;
        t.Prazo = Prazo;
    }
}
```

Arquivo: `src/PortalTarefas.Web/ViewModels/TarefaViewModels.cs` (commit `e6a3427`)

```
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.ViewModels;

/// <summary>Dados exibidos no resumo de uma tarefa (partial _TarefaResumo).</summary>
public sealed record TarefaResumoViewModel(int Id, string Titulo, SituacaoTarefa Situacao,
    PrioridadeTarefa Prioridade, DateOnly? Prazo, bool Vencida)
{
    public static TarefaResumoViewModel De(Tarefa t, DateOnly hoje) =>
        new(t.Id, t.Titulo, t.Situacao, t.Prioridade, t.Prazo, t.EstaVencida(hoje));
}

public sealed class TarefaListaViewModel
{
    public SituacaoTarefa? Situacao { get; init; }
    public string? Busca { get; init; }
    public IReadOnlyList<TarefaResumoViewModel> Itens { get; init; } = [];
}

public sealed class TarefaDetalhesViewModel
{
    public required TarefaResumoViewModel Resumo { get; init; }
    public string? Descricao { get; init; }
    public DateTimeOffset CriadaEm { get; init; }
}

public sealed class TarefaEdicaoViewModel
{
    public int Id { get; init; }
    public required TarefaInputModel Entrada { get; init; }
}
```

Arquivo: `src/PortalTarefas.Web/Filtros/RegistroAcaoFilter.cs` (commit `e6a3427`)

```
using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Controllers;
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc.Infrastructure;

namespace PortalTarefas.Web.Filtros;

/// <summary>
/// Filtro de ação: registra ação, duração, validade do ModelState e tipo/status do resultado.
/// Não altera o fluxo e não contém regra de negócio — a validação continua no controller via ModelState.
/// </summary>
public sealed class RegistroAcaoFilter(ILogger<RegistroAcaoFilter> logger) : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate
next)
    {
        var acao = context.ActionDescriptor is ControllerActionDescriptor cad
            ? $"{cad.ControllerName}.{cad.ActionName} [{context.HttpContext.Request.Method}]"
            : context.ActionDescriptor.DisplayName;
        var inicio = Stopwatch.GetTimestamp();
        logger.LogInformation("[Filtro] Executando {Acao} (ModelState válido: {Valido})", acao,
context.ModelState.IsValid);

        var executado = await next();

        var resultado = executado.Result switch
        {
            ViewResult v => $"View({v.ViewName ?? "padrão"}) status {v.StatusCode ?? 200}",
            RedirectToActionResult r => $"Redirect -> {r.ActionName}",
            IStatusCodeActionResult s => $"{executado.Result.GetType().Name} status {s.StatusCode}",
            null when executado.Exception is not null => $"Exceção {executado.Exception.GetType().Name}",
            _ => executado.Result?.GetType().Name ?? "nenhum"
        };
        logger.LogInformation("[Filtro] Concluído {Acao} -> {Resultado} em {DuracaoMs:F1} ms",
            acao, resultado, Stopwatch.GetElapsedTime(inicio).TotalMilliseconds);
    }
}
```

Arquivo: `src/PortalTarefas.Web/ViewComponents/TarefasVencidasViewComponent.cs` (commit `e6a3427`)

```
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Servicos;
using PortalTarefas.Web.ViewModels;

namespace PortalTarefas.Web.ViewComponents;

/// <summary>Componente reutilizável que mostra tarefas vencidas no layout.</summary>
public sealed class TarefasVencidasViewComponent(ITarefaServico servico, TimeProvider relogio) :
ViewComponent
{
    public async Task<IViewComponentResult> InvokeAsync(int maximo = 5)
    {
        var hoje = DateOnly.FromDateTime(relogio.GetLocalNow().Date);
        var vencidas = await servico.ListarVencidasAsync(hoje, HttpContext.RequestAborted);
        return View(vencidas.Take(maximo).Select(t => TarefaResumoViewModel.De(t, hoje)).ToList());
    }
}
```

Arquivo: `src/PortalTarefas.Web/Views/Shared/_Layout.cshtml` (commit `e6a3427`)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>@ViewData["Title"] · Portal de Tarefas</title>
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
    <header class="topo">
        <a class="marca" asp-controller="Tarefas" asp-action="Index">Portal de Tarefas</a>
        <nav>
            <a asp-controller="Tarefas" asp-action="Index">Lista</a>
            <a asp-controller="Tarefas" asp-action="Create">Nova tarefa</a>
        </nav>
    </header>
    <div class="conteudo">
        <main>
            @if (TempData["Mensagem"] is string mensagem)
            {
                <div class="aviso" role="status">@mensagem</div>
            }
            <h1>@ViewData["Title"]</h1>
            @RenderBody()
        </main>
        <aside>
            @await Component.InvokeAsync("TarefasVencidas", new { maximo = 5 })
        </aside>
    </div>
    <footer>Desenvolvimento Web com .NET · ASP.NET Core 10</footer>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

Arquivo: `src/PortalTarefas.Web/Views/Shared/_TarefaResumo.cshtml` (commit `e6a3427`)

```
@model TarefaResumoViewModel
<article class="cartao @(Model.Vencida ? "vencida" : "")">
    <h3><a asp-controller="Tarefas" asp-action="Details" asp-route-id="@Model.Id">#@Model.Id ·
@Model.Titulo</a></h3>
    <p>
        <span class="selo situacao-@Model.Situacao">@Model.Situacao</span>
        <span class="selo prioridade-@Model.Prioridade">Prioridade @Model.Prioridade</span>
        @if (Model.Prazo is { } prazo)
        {
            <span class="prazo">Prazo: @prazo.ToString("dd/MM/yyyy")</span>
        }
        @if (Model.Vencida)
        {
            <strong class="alerta">Vencida</strong>
        }
    </p>
</article>
```

Arquivo: `src/PortalTarefas.Web/Views/Shared/_TarefaCampos.cshtml` (commit `e6a3427`)

```
@model TarefaInputModel
@* Campos compartilhados entre Create e Edit. Os Tag Helpers geram name/id/value e mensagens de
validação. *@
<div class="campo">
    <label asp-for="Titulo"></label>
```

```
    <input asp-for="Titulo" />
    <span asp-validation-for="Titulo" class="erro"></span>
</div>
<div class="campo">
    <label asp-for="Descricao"></label>
    <textarea asp-for="Descricao" rows="3"></textarea>
    <span asp-validation-for="Descricao" class="erro"></span>
</div>
<div class="linha">
    <div class="campo">
        <label asp-for="Prioridade"></label>
        <select asp-for="Prioridade" asp-items="Html.GetEnumSelectList<PrioridadeTarefa>()"></select>
        <span asp-validation-for="Prioridade" class="erro"></span>
    </div>
    <div class="campo">
        <label asp-for="Situacao"></label>
        <select asp-for="Situacao" asp-items="Html.GetEnumSelectList<SituacaoTarefa>()"></select>
    </div>
    <div class="campo">
        <label asp-for="Prazo"></label>
        <input asp-for="Prazo" type="date" />
        <span asp-validation-for="Prazo" class="erro"></span>
    </div>
</div>
```

Arquivo: `src/PortalTarefas.Web/Views/Shared/Components/TarefasVencidas/Default.cshtml` (commit `e6a3427`)

```
@model IReadOnlyList<TarefaResumoViewModel>
<section class="vencidas" aria-labelledby="titulo-vencidas">
    <h2 id="titulo-vencidas">Tarefas vencidas (@Model.Count)</h2>
    @if (Model.Count == 0)
    {
        <p>Nenhuma tarefa vencida.</p>
    }
    else
    {
        <ul>
            @foreach (var t in Model)
            {
                <li><a asp-controller="Tarefas" asp-action="Details" asp-route-id="@t.Id">@t.Titulo</a> ·
@t.Prazo?.ToString("dd/MM")</li>
            }
        </ul>
    }
</section>
```

Arquivo: `src/PortalTarefas.Web/Views/Tarefas/Index.cshtml` (commit `e6a3427`)

```
@model TarefaListaViewModel
@{ ViewData["Title"] = "Tarefas"; }
<form method="get" asp-action="Index" class="filtros">
    <input name="busca" value="@Model.Busca" placeholder="Buscar no título" aria-label="Buscar no título"
/>
    <select name="situacao" asp-items="Html.GetEnumSelectList<SituacaoTarefa>()" aria-label="Situação">
        <option value="">Todas as situações</option>
    </select>
    <button type="submit">Filtrar</button>
</form>
@if (Model.Itens.Count == 0)
{
    <p class="vazio">Nenhuma tarefa encontrada.</p>
}
@foreach (var item in Model.Itens)
{
    <partial name="_TarefaResumo" model="item" />
}
```

Arquivo: `src/PortalTarefas.Web/Views/Tarefas/Details.cshtml` (commit `e6a3427`)

```
@model TarefaDetalhesViewModel
@{ ViewData["Title"] = "Detalhes da tarefa"; }
<partial name="_TarefaResumo" model="Model.Resumo" />
<dl>
    <dt>Descrição</dt><dd>@(Model.Descricao ?? "(sem descrição)")</dd>
    <dt>Criada em</dt><dd>@Model.CriadaEm.ToLocalTime().ToString("dd/MM/yyyy HH:mm")</dd>
</dl>
<p>
    <a class="botao" asp-action="Edit" asp-route-id="@Model.Resumo.Id">Editar</a>
    <a asp-action="Index">Voltar à lista</a>
</p>
```

Arquivo: `src/PortalTarefas.Web/Views/Tarefas/Create.cshtml` (commit `e6a3427`)

```
@model TarefaInputModel
@{ ViewData["Title"] = "Nova tarefa"; }
<form asp-action="Create" method="post" novalidate>
    <div asp-validation-summary="ModelOnly" class="erro"></div>
    <partial name="_TarefaCampos" model="Model" />
    <button type="submit">Cadastrar</button>
    <a asp-action="Index">Cancelar</a>
</form>
```

Arquivo: `src/PortalTarefas.Web/Views/Tarefas/Edit.cshtml` (commit `e6a3427`)

```
@model TarefaEdicaoViewModel
@{ ViewData["Title"] = $"Editar tarefa #{Model.Id}"; }
<form asp-action="Edit" asp-route-id="@Model.Id" method="post" novalidate>
    <div asp-validation-summary="ModelOnly" class="erro"></div>
    @* O prefixo "Entrada" faz os campos serem enviados como Entrada.Titulo, Entrada.Prazo... *@
    <partial name="_TarefaCampos" for="Entrada" />
    <button type="submit">Salvar</button>
    <a asp-action="Details" asp-route-id="@Model.Id">Cancelar</a>
</form>
```

Apêndice B – Capturas de tela

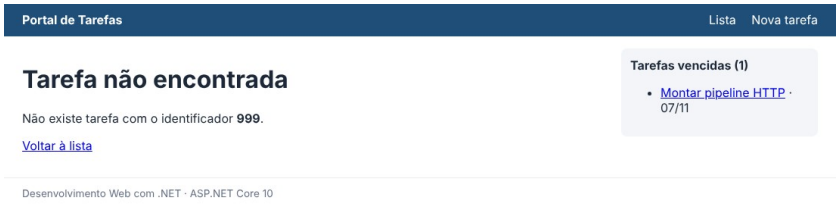


Figura B1 – Identificador inexistente (status 404, view própria)

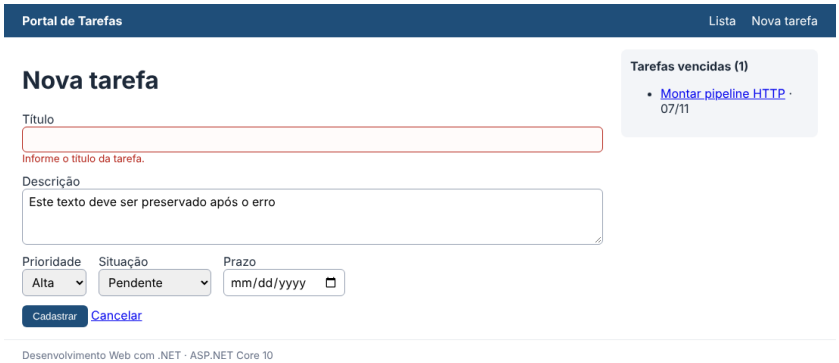


Figura B2 – Cadastro com título vazio; descrição preservada

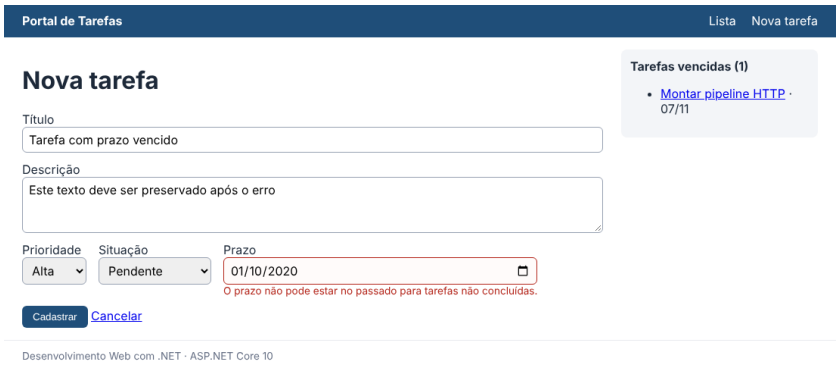


Figura B3 – Prazo no passado para tarefa não concluída

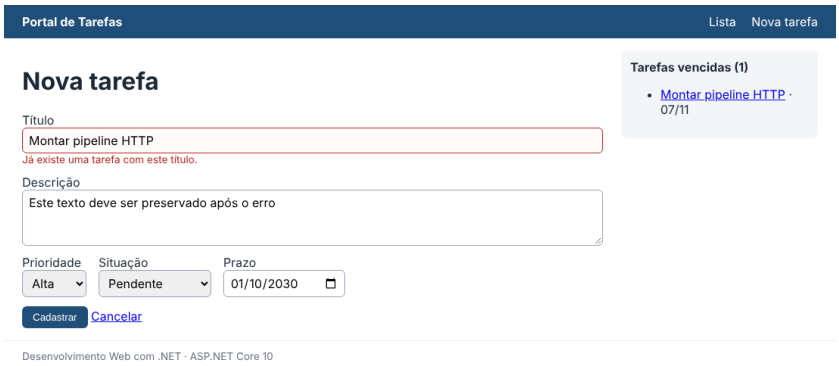


Figura B4 – Validação assíncrona: título duplicado

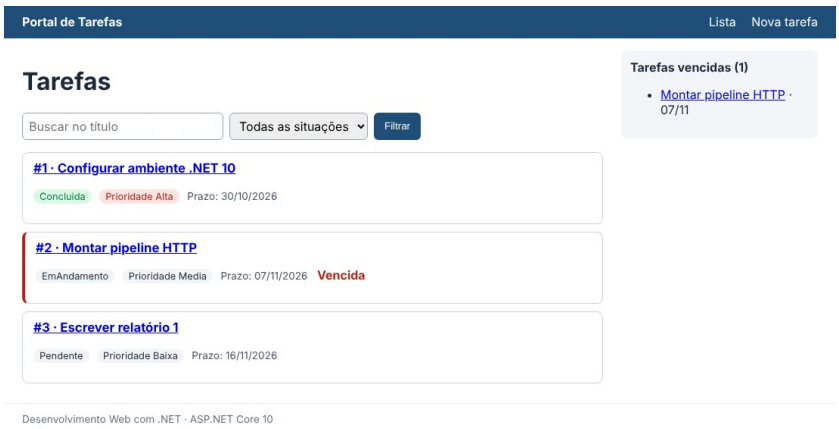


Figura B5 – Lista com partial de resumo e View Component de vencidas

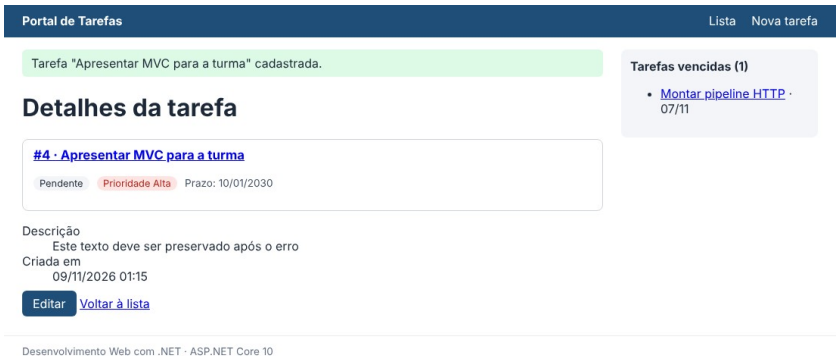


Figura B6 – Cadastro válido redirecionado para Detalhes (PRG)

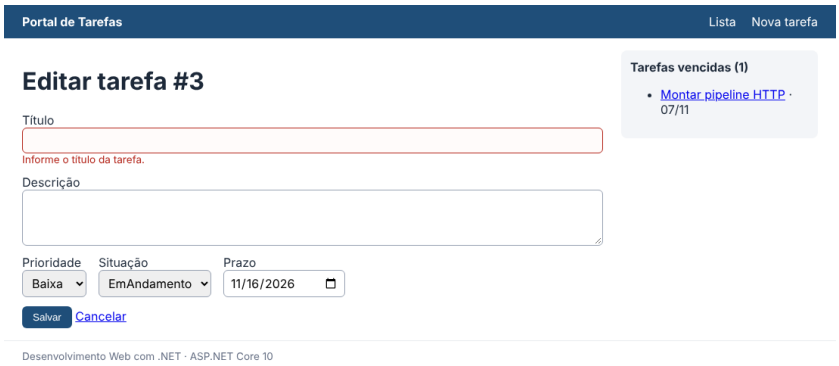


Figura B7 – Edição com erro de validação

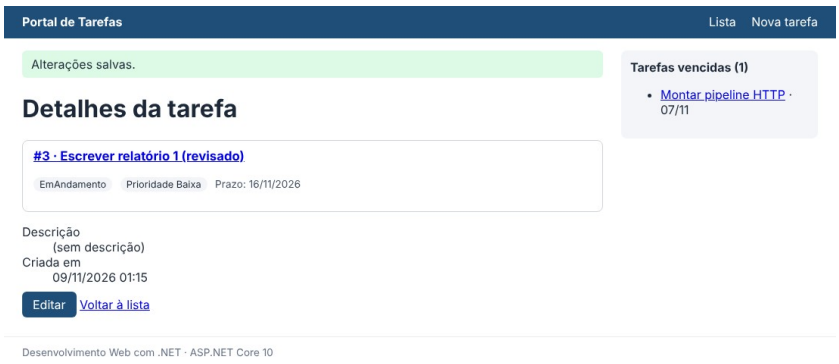


Figura B8 – Edição salva